Software Engineering Assignment

Ethan Witwer, Linköping University

June 2026

1 Introduction

Broadly, my research is in the field of security (specifically network security) and privacy. My PhD project is to create defenses against traffic analysis.

Most online applications now use network-level encryption to protect users against eavesdropping by network observers. This makes it so that, for example, it is safe to send credit card data during an online purchase (well, assuming the vendor is trusted and the TLS certificate is authentic) — nobody on the network path will be able to see the data. *Traffic analysis* is a way to thwart this protection, using heuristics and machine learning algorithms to recognize patterns in encrypted network flows, that correspond to some known content. This can be used to identify the websites a user visits, the videos they watch, and more.

Interest in this research area has increased during the past few years, resulting in a number of publications that present better attacks, attacker models, and defenses. Unfortunately, most proposed defenses require too much additional bandwidth or latency (*overhead*), have substantive negative impacts on user experience (*QoE*), or are difficult to deploy. Due in part to this, most online applications remain unprotected. My goal is to address this issue by developing solutions with greater promise in the real world, and I aim to publish source code along with all of my papers (and tools that anyone can use to protect themselves).

2 Lecture Principles

(I) One topic that is interesting to discuss in relation to my research area is *verification* (determining whether a production is being built correctly). In the lectures, it was discussed that the three primary techniques for verification are static analysis (code review, tools that check source code, etc.), dynamic analysis (tests on a compiled software product), and process evaluation techniques.

A key point from the lectures is that it is difficult to apply these sorts of techniques to AI models, since it is hard to analyze a black box systematically, and models are non-deterministic. This is also the case in my research area, but not in exactly the same way, since I create defenses against classification models, aiming to degrade them rather than make them work.

One obvious way to test defenses, and one that is employed in most studies, is to pick some state-of-the-art classification model(s), apply the defense strategies under study to a dataset of network traces from website crawls, video streams, etc. (depending on the problem at hand), and run the model(s) against that dataset. This generalizes horribly, of course, providing only an indication of a defense's adequacy: new attacks often defeat defenses previously thought to provide strong protection, and defenses may not work as well on datasets they haven't been tested on, or in the real world where much more data exists.

To add to the mess, information leakage can occur in other ways. One example is video streaming. Say that we want N videos on a platform to have the same traffic patterns, so that a network observer can tell which anonymity set a video belongs to, but not exactly which video it is. If the platform provides view counts, like counts, etc. — as many platforms do — this makes it more likely that certain videos in the anonymity set have been watched, unless the anonymity set is constructed extremely carefully. Also, correlation attacks could be performed, among other possibilities I may not have even thought of.

Some solutions include having a clearly defined threat model and not attempting to address threats outside of it (a form of requirement definition); creating defenses with certain intended properties, even if these are statistical rather than deterministic, and evaluating them against those properties; and testing against multiple datasets in different ways and with different attacks. In short, it can help to employ property-based testing, or a variant of it.

(II) *Behavioral software engineering* is also interesting to think about. In the lectures, there was some discussion of cognitive bias and how it affects software engineers/organizations. For example, cognitive bias can lead to higher defect rates, incorrectly believing that some piece of code is correct, or insufficient code review before merging changes.

Since the purposes and extent of traffic analysis in the wild are largely unknown (though there are indications), a number of developers are wary of implementing protection when it requires overhead or QoE trade-offs. Also, developers — even those with great knowledge about traffic analysis — sometimes operate under false assumptions about what implementing protection would entail. For example, it was widely believed for a time that defenses based only on injecting additional packets (padding) into the network, called padding-only defenses, did not introduce delay

in website downloads via mix networks. My first research project was to provide evidence to refute this belief.

In short, domain-specific implications of cognitive bias and other psychological effects should be considered when developing software (and in my research area). This is something that I do think about rather often, since it significantly affects not only whether software developers are willing to implement defenses but also whether users are willing to turn them on. But it may help to adopt a more systematic approach, beginning with an understanding of different types of cognitive bias and working from there.

3 Guest Lecture Principles

(I) During the discussion of what will become of software engineers with the increasing prevalence of LLMs (Per Lenberg), it was discussed that removing administrative overhead ”increases the need to design a sustainable balance between cognitive intensity, variation, and recovery”.

This is especially true in my research area, since a lot of careful decisions often need to be made during the development of defenses. These decisions lead, in turn, to more general thought that can result in more holistic defense solutions and improve the overall quality of the work. My experience of coding with LLMs is that things go so quickly that there is constantly code to review and decisions to be made, resulting in significant cognitive load, leading to perceived pressure to make decisions quickly, and reducing the insights that come during slow, methodic manual development.

One way to prevent this may be to spend more time on manual review, do different types of review, and/or allocate time specifically for thought, and perhaps think through different aspects of the system in a specific way (while, of course, leaving room for creativity). Another solution may be to have multiple people collaborate on or discuss smaller problems, even when this is not strictly necessary. These are only spontaneous ideas, though, and it is hard to fully address the issue. On the flip side, coding with LLMs does remove certain issues; it could also be argued that reducing focus on specific implementation details allows for the more high-level thought that I implicitly claim is important. My general conclusion is that time for meaningful thought is crucial.

(II) An aspect of requirements engineering mentioned during the lecture based on Julian Frattini’s slides is that requirements must be *free from defects*: unambiguous, complete, specific, and relevant. This is closely related to my discussion of verification above. Testing is often a vague process, at least in part because

requirements are often vaguely formulated.

For example, defenses often have some variation of the vague goal of "randomizing traffic patterns." In some cases, this leads to solutions that have some sort of intuition behind them but are still not entirely based on principles — especially in the case of web traffic, which is at least at some level a black box — and whose claimed intuitiveness could just as well have been come up with after the fact.

Since I am currently focused on video traffic, which is much easier to reason about, it's easy for me to say that a goal of randomness should be made more specific by adding some sort of statistical reasoning; otherwise, the goal should be deterministic. In any case, it is at least a good idea to think about how goals should be formulated in my research area, since the focus is often on making things (such as degree of protection) good enough for publication instead. I have done this in my recent projects, including with my students, by defining goals mathematically.

4 Data Scientists, Software Engineers, and AI Engineers

What differences do the chapters describe between data scientists and software engineers? These two roles are contrasted clearly and directly in the first chapter. While data scientists typically have a strong background in machine learning and/or statistics and focus on building models and working with data in different ways (a bit more scientifically), software engineers focus on building products that meet customers' needs within specified time and resource constraints.

Apparently, typical software engineering curricula cover requirements engineering, software design, quality assurance, etc. with some other topics thrown in. This seems like a reasonable assumption to make about dedicated software engineering degrees, but I can't relate myself, since I did my undergraduate degree in Computer Science (as did most software engineers I know), which has a more broad focus with less specific focus on the typical activities of software engineers.

Do you agree with this distinction? Why or why not? The distinction is a bit oversimplified, but I think it's true to some extent in most cases. My experience working as a software engineer is that the main focus is customer satisfaction and the things needed to achieve that, and my fairly superficial knowledge of the life of data scientists suggests that they do have a more statistics/machine learning-based approach, focusing on models. But there are, of course, exceptions, and I worked at a software company where (at least in the teams I worked with) AI was not needed or used at all, and I may have a different perspective otherwise.

The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area? This is primarily applicable to the attack side of traffic analysis. Classification

models are often tested in isolation to demonstrate that they can differentiate traffic from different web pages, videos, etc.; but where would they be executed in practice? On an ISP's routers, or somewhere else? And how would training data be acquired, predictions be stored, traffic flows be matched with users, and so on? These things depend largely on the threat model, and there are many valid answers, but the answers affect how the system might work in the real world.

Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills? Explain. The book's perspective seems to be that specialization will not be erased but that more system-level understanding is needed ("T-shaped team members"). Though I think that systems thinking is very important, especially from a security standpoint, I am hesitant to fully agree with this. Depending on the project (despite what many seem to think, there are still plenty of use cases for software that doesn't have AI), data science might not really be applicable at all. Also, it is conceivable that engineers working on a small, well-defined module in a system might not, in practice, have any need (or external pressure) to learn too much about other components of the system; and data scientists may be able to make a model work as it should within a larger system by adhering to strict specifications, even if they may be hard to come by, rather than understanding the rest of the system. In short, it depends on the project, its size, the scope of what an individual/team is doing, and potentially other factors. With proper modularity and interface specifications, I think the need for teams to learn about the rest of the system they're working on can reasonably be minimized, though perhaps not eliminated.

Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct? I would say that AI Engineering aligns most closely with MLOps/model engineering. If we think of AI Engineering as involving the creation of science-engineering loops, as in the lectures, this is achieved via machine learning pipelines. I would, however, say that AI Engineering is partly distinct from MLOps/model engineering due to the greater focus on observation and refinement (a feedback loop) than pure engineering effort.

How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles? They push the focus of both roles towards integration and interaction with existing systems rather than creation of custom models, making it easier for people in the more rigidly defined software engineering role to deal with AI components of systems, while data scientists perform less of their traditional activities. At the same time, handling interfaces becomes more unwieldy and challenging (particularly if multiple models/AI systems are used), and failure modes change, which also affects the focus of the two roles, as issues such as prompt injection must be accounted for.

5 Paper Analysis

Paper 1: Themes of Building LLM-based Applications for Production: A Practitioner's View (CAIN 2025)

Core ideas and their SE importance: The paper takes videos of practitioners developing LLM-based systems, analyzes transcripts to collect the topics they discuss, and maps them into themes; the goal is to provide insights into where priorities lie in LLM system development.

The main point is that development of systems that employ LLMs has its own challenges beyond traditional ML-based system development. The themes include, from the analysis: architecture and design; model capabilities and techniques; tools, infrastructure, and LLM providers; evaluation; risks and ethics; monitoring and observability; costs; and output verification. The authors also describe the topics under each theme and run a co-occurrence analysis (for example, RAG co-occurs with evaluation, prompt engineering, and security). These things are important because they show where more effort is needed so that developers can realize their priorities, and it can show where priorities may be out of order.

Relation to my research: The paper's ideas fall into my research, and my research falls into the paper's framework to a certain extent. I think the most useful way to apply the paper's methodology to my research would be to do a similar analysis to get a better idea of what people think is important in traffic analysis, and potentially find more systematic ways to address those concerns or reprioritize. A variation could also be done on a smaller scale, for example on myself. My research could be relevant to the topics in the paper also: for example, security is a topic mentioned in the paper, and traffic analysis is a part of that. This also begs the question of what practicioners think that the different topics really entail.

Integration into a larger AI-intensive project: One example of a project could be a control panel used to operate a complex physical system, for example, a manufacturing warehouse. Due to the complexity and different components, different types of AI models are needed, and this could result in a massive amount of glue code, complexity explosion, and other issues. This paper's methods could be use to analyze the priorities of the team(s) working on (and even using) the software. Ideally, this should be done during both initial planning and development stages. This could reduce the daunting nature of the development task by helping to organize priorities, make them more specific, and refine them. My research fits into this project because it would not be desirable for someone with access to the communication link(s) between different parts of the system, such as controllers in the warehouse and the control panel software, to be able to discern sensitive information.

Adaptation of your research: As I mentioned briefly, the most realistic way to adapt my own research projects, which I mostly do on my own apart from frequent

brainstorming and some limited technical collaboration with my supervisors, would be to analyze the topics I/we discuss to decide which priorities need more attention and how to prioritize better.

Paper 2: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering (NeurIPS 2024)

Core ideas and their SE importance: This paper proposes the idea that language model (LM) agents can be considered a class of end user, with unique needs and abilities. They should be treated as such, and care should be put into designing the computer-agent interface. The paper explores how interface design affects LM agents' performance and proposes a system called SWE-agent, with an improved ability to solve software engineering tasks. The importance of the paper to software engineering could not be more clear: beyond SWE-agent, their focus is on the code around an LM agent, not just the agent itself, and the insights they provide are very relevant.

Relation to my research: Despite the fact that the paper goes well beyond prompt design, I think the main insights that can be applied to my research are the factors that make certain interface designs work. The systems I build and work with don't use LMs, but LMs are often used for the development itself as well as brainstorming and other tasks, and it is especially important to be efficient and facilitate for LMs as much as possible when dealing with relatively niche topics such as mine. Traffic analysis could also be considered when designing agent-computer interfaces (if network communication occurs).

Integration into a larger AI-intensive project: For the sake of coherence, I'll use the same fantasy project as above: a control panel system for a manufacturing warehouse, with many distinct AI components. The paper's ideas can be applied to improve the performance of the models by improving the agent-computer interface, both between agents and the physical devices being controlled and between agents and the control panel software. This might also shape the interface that a human user sees. As far as my research goes, if any of the data to/from an agent is passed over some communication link, it could be intercepted, so traffic analysis could also be considered in agent-computer interface design.

Adaptation of your research: One possible application would be to rethink the way data is provided to traffic analysis attack models. Often, data is provided as one or a few time series, but some models incorporate different types of data, usually still using only one or a few input series. More targeted data could be provided to the classification models as long as it is presented in a suitable way.

6 Evidence, Originality, and GenAI Transparency

Apart from the course materials and book chapters linked in the assignment description, I only consulted the two papers named above:

- A. Mailach, S. Simon, J. Dorn, and N. Siegmund, ”Themes of Building LLM-based Applications for Production: A Practitioner’s View,” in Proc. IEEE/ACM 4th Int. Conf. on AI Engineering – Software Engineering for AI (CAIN), Ottawa, Canada, 2025.
- J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, ”SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024.

I used GenAI tools to summarize the book chapters and papers I read (but not any course materials) — as a supplement to my own reading — and to find suitable papers for the paper analysis section. I did not generate any text, ask for writing suggestions, or review my writing with GenAI, nor did I do spell-checking, etc. My answers reflect only my own opinions and thoughts.